

Reporte de
PRÁCTICA 02



SEMAFOROS

**CONCURRENCIA Y
PARALELISMO**

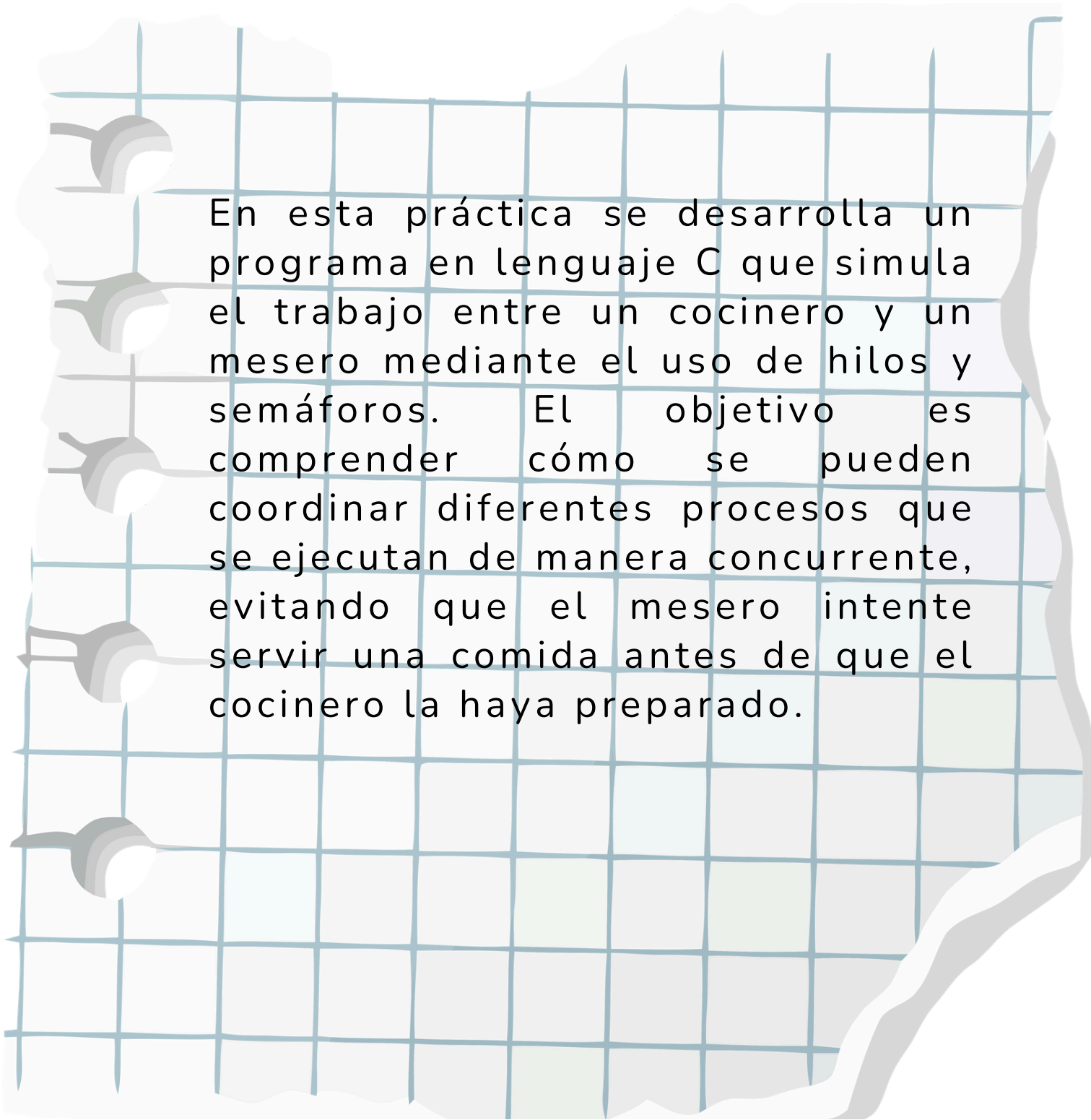
**INGENIERIA EN TECNOLOGIA
DE SOFTWARE**

ALAN CAMPOS SUAREZ

UNIVERSIDAD AUTONOMA DE CAMPECHE



INTRODUCCION



En esta práctica se desarrolla un programa en lenguaje C que simula el trabajo entre un cocinero y un mesero mediante el uso de hilos y semáforos. El objetivo es comprender cómo se pueden coordinar diferentes procesos que se ejecutan de manera concurrente, evitando que el mesero intente servir una comida antes de que el cocinero la haya preparado.



DESARROLLO

```
C programa.c
1  /*
2  * Cabecera del programa:
3  * Programa: Cocinero y Mesero con Semaforos
4  * Autor: Alan Campos
5  * Fecha: 08/09/2026
6  * Descripcion: Simula la preparacion y servicio de comida
7  *               utilizando hilos y un semaforo para sincronizarlos.
8  */
9
10 #include <pthread.h> // Permite trabajar con hilos
11 #include <semaphore.h> // Permite utilizar semaforos
12 #include <stdio.h> // Funciones de entrada y salida
13 #include <time.h> // Funciones relacionadas con el tiempo
14 #include <unistd.h> // Permite utilizar sleep()
15
```

El programa utiliza la biblioteca pthread para crear dos hilos: uno para el cocinero y otro para el mesero. También utiliza un semáforo mediante la biblioteca semaphore.h para sincronizar las acciones de ambos hilos.



El hilo del cocinero prepara 10 comidas y, después de preparar cada una, utiliza `sem_post()` para avisar al mesero que puede servirla. Por otro lado, el hilo del mesero utiliza `sem_wait()` para esperar hasta recibir la señal del cocinero antes de servir cada comida. Al finalizar ambos hilos, el programa muestra el valor final del contador.

```
// Crea el hilo que ejecutara la funcion cocinar
pthread_create (&cocinero, NULL, *cocinar, NULL);

// Crea el hilo que ejecutara la funcion servir
pthread_create (&mesero, NULL, *servir, NULL);

// Espera a que termine el hilo del cocinero
pthread_join(cocinero, NULL);

// Espera a que termine el hilo del mesero
pthread_join(mesero, NULL);

printf("Contador %d \n", counter); // Muestra el valor final del contador

return 0; // Finaliza el programa
```

```
// Recorre el ciclo 10 veces para preparar la comida
for (int i = 0; i < NR_LOOP; i++)
{
    // sem_wait(&sem1);
    // counter++;
    // sem_post(&sem1);

    printf("COCINERO: Comida preparada \n"); // Muestra que la comida esta lista

    sem_post(&sem1); // Libera el semaforo para avisar al mesero

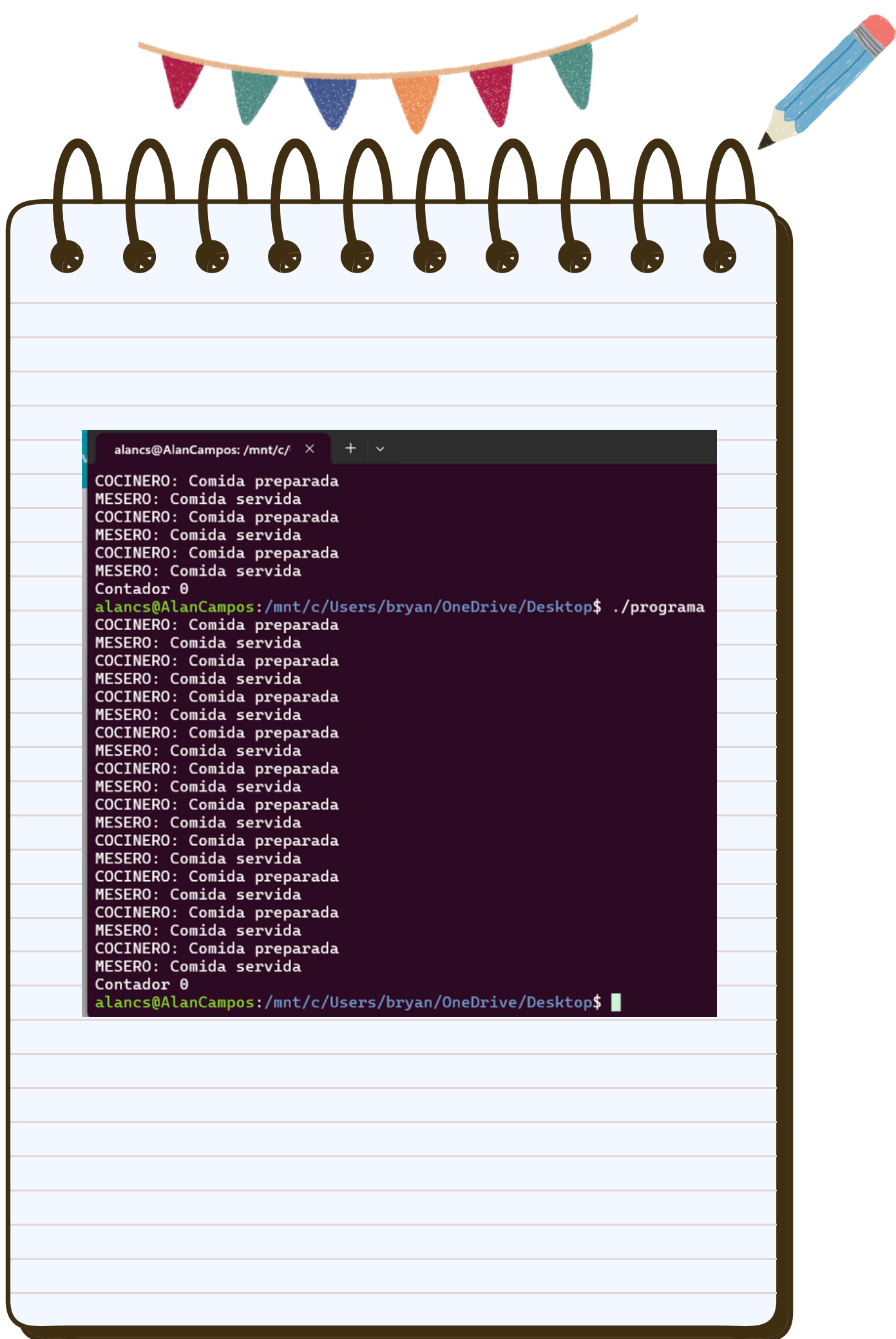
    sleep(1); // Espera 1 segundo antes de preparar otra comida
}
```



```
alancs@AlanCampos:/mnt/c/Users/bryan/OneDrive/Desktop$ ./programa
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
Contador 0
```

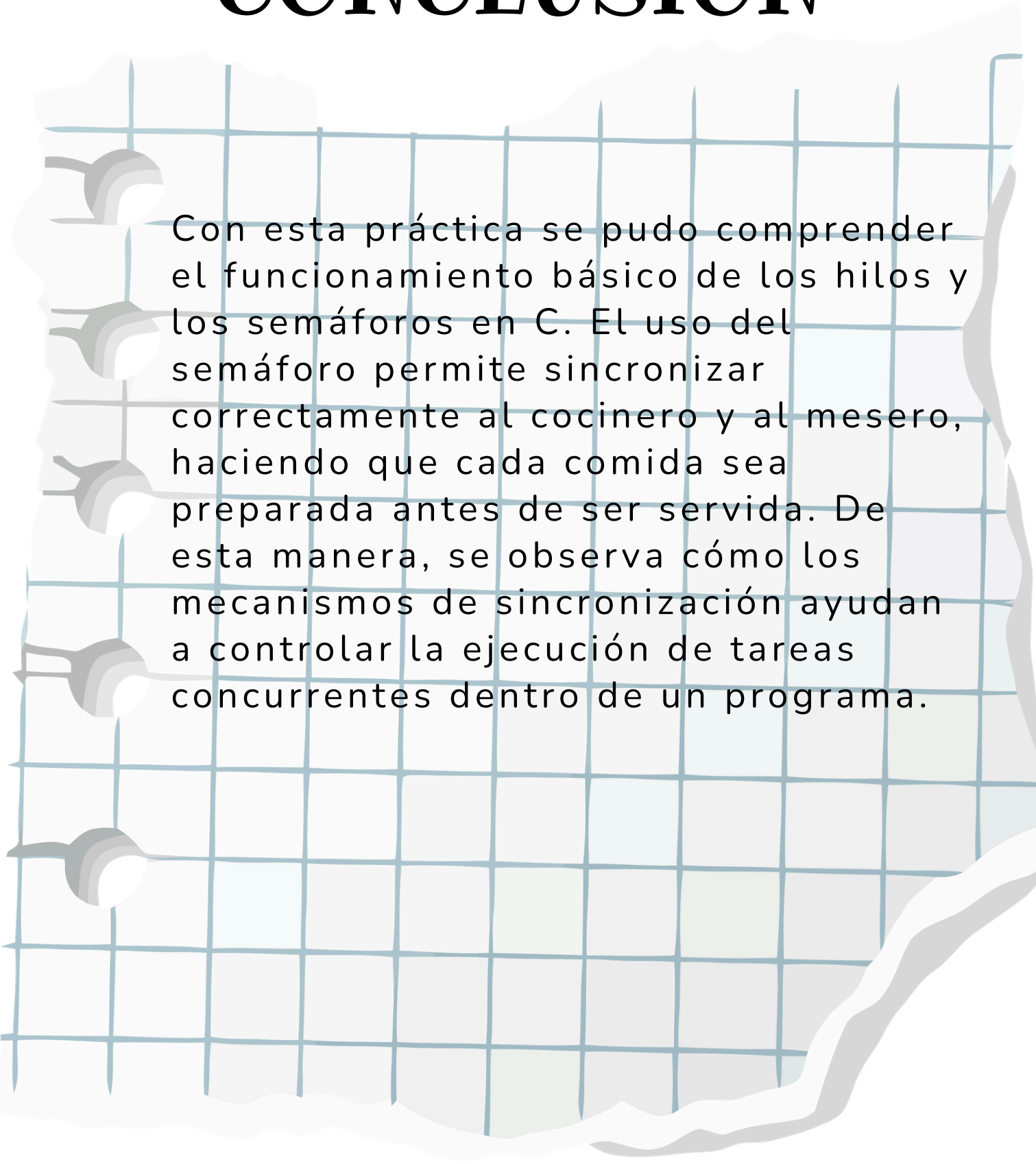
```
alancs@AlanCampos:/mnt/c/Users/bryan/OneDrive/Desktop$ gcc programa.c -o programa
alancs@AlanCampos:/mnt/c/Users/bryan/OneDrive/Desktop$ ./programa
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
COCINERO: Comida preparada
MESERO: Comida servida
Contador 0
```

compilar y ejecutar





CONCLUSION



Con esta práctica se pudo comprender el funcionamiento básico de los hilos y los semáforos en C. El uso del semáforo permite sincronizar correctamente al cocinero y al mesero, haciendo que cada comida sea preparada antes de ser servida. De esta manera, se observa cómo los mecanismos de sincronización ayudan a controlar la ejecución de tareas concurrentes dentro de un programa.